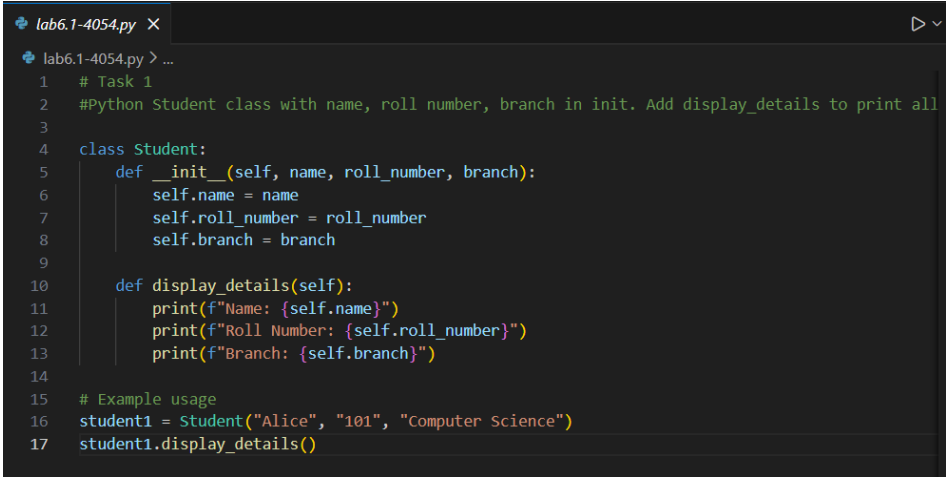


SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Code	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week3 – Wednesday	Batch No	23CSBTB47B
Name	Aelishala Manoj	Hall Ticket No	2303A54054
AssignmentNumber:6.3			

Q.No.	Question
1	Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals Lab Objectives • To explore AI-powered auto-completion features for core Python constructs such as classes, loops, and conditional statements. • To analyze how AI tools suggest logic for object-oriented programming and control structures. • To evaluate the correctness, readability, and completeness of AI-generated Python code. Lab Outcomes (LOs) After completing this lab, students will be able to: • Use AI tools to generate and complete Python class definitions and methods. • Understand and assess AI-suggested loop constructs for iterative tasks. • Generate and evaluate conditional statements using AI-driven prompts. • Critically analyze AI-assisted code for correctness, clarity, and efficiency. Task Description #1: Classes (Student Class) Scenario You are developing a simple student information management module. Task • Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class. • The class should include attributes such as name, roll number, and branch. • Add a method <code>display_details()</code> to print student information. • Execute the code and verify the output. • Analyze the code generated by the AI tool for correctness and clarity. Expected Output #1 • A Python class with a constructor (<code>__init__</code>) and a <code>display_details()</code> method. • Sample object creation and output displayed on the console. • Brief analysis of AI-generated code. Prompt : Create simple Python Student class with name, roll number, branch in init. Add <code>display_details</code> to print all info. Include sample object and output. <pre>lab6.1-4054.py X lab6.1-4054.py > ... 1 # Task 1 2 #Python Student class with name, roll number, branch in init. Add display_details to print all 3 4 class Student: 5 def __init__(self, name, roll_number, branch): 6 self.name = name 7 self.roll_number = roll_number 8 self.branch = branch 9 10 def display_details(self): 11 print(f"Name: {self.name}") 12 print(f"Roll Number: {self.roll_number}") 13 print(f"Branch: {self.branch}") 14 15 # Example usage 16 student1 = Student("Alice", "101", "Computer Science") 17 student1.display_details()</pre>

Output :

```
Name: Alice
Roll Number: 101
Branch: Computer Science
```

Explanation:

The Student class is defined with an `__init__` method that initializes the name, roll number, and branch attributes. The `display_details` method prints out the student's information in a formatted manner.

Task Description #2: Loops (Multiples of a Number)

Scenario

You are writing a utility function to display multiples of a given number.

Task

- Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a loop.
- Analyze the generated loop logic.
- Ask the AI to generate the same functionality using another controlled looping structure (e.g., while instead of for).

Expected Output #2

- Correct loop-based Python implementation.
- Output showing the first 10 multiples of a number.
- Comparison and analysis of different looping approaches.

```
lab6.1-4054.py X
lab6.1-4054.py > display_multiples
19 # Task 2
20 # writing a utility function to display multiples of a given number.
21
22 def display_multiples(number, count):
23     for i in range(1, count + 1):
24         print(f"{number} * {i} = {number * i}")
25
26 # Example usage
27 print("\nMultiples of 5:")
28 display_multiples(5, 10) # Display first 10 multiples of 5
```

Output :

```
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
```

Explanation :

The function `display_multiples` takes two parameters: `number` and `count`. It uses a for loop to iterate from 1 to `count` (inclusive) and prints the multiples of the given number.

Task Description #3: Conditional Statements (Age Classification)

Scenario

You are building a basic classification system based on age.

Task

- Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups (e.g., child, teenager, adult, senior).
- Analyze the generated conditions and logic.
- Ask the AI to generate the same classification using alternative conditional structures (e.g., simplified conditions or dictionary-based logic).

Expected Output #3

- A Python function that classifies age into appropriate groups.
- Clear and correct conditional logic.
- Explanation of how the conditions work.

```
lab6.1-4054.py •
lab6.1-4054.py > ...
32 # Task 3
33 # building a basic classification system based on age.
34 def classify_age(age):
35     if age < 0:
36         return "Invalid age"
37     elif age <= 12:
38         return "Child"
39     elif age <= 19:
40         return "Teenager"
41     elif age <= 59:
42         return "Adult"
43     else:
44         return "Senior Citizen"
45 # Example usage
46 ages = [10, 15, 25, 65, -5]
47 for age in ages:
48     category = classify_age(age)
49     print(f"Age: {age}, Category: {category}")
50 print("\nAge Classification:")
```

Output :

```
Age Classification:
Age: 10, Category: Child
Age: 15, Category: Teenager
Age: 25, Category: Adult
Age: 65, Category: Senior Citizen
Age: -5, Category: Invalid age
```

Explanation:

The `classify_age` function takes an age as input and classifies it into categories: Child, Teenager, Adult, and Senior Citizen based on the specified age ranges. It also handles invalid ages.

Task Description #4: For and While Loops (Sum of First n Numbers)

Scenario

You need to calculate the sum of the first n natural numbers.

Task

- Use AI assistance to generate a `sum_to_n()` function using a for loop.
- Analyze the generated code.
- Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

Expected Output #4

- Python function to compute the sum of first n numbers.
- Correct output for sample inputs.
- Explanation and comparison of different approaches.

```
lab6.1-4054.py X
lab6.1-4054.py > ...
52
53 # Task 4
54 #calculate the sum of the first n natural numbers.
55
56 def sum_of_natural_numbers(n):
57     if n < 1:
58         return 0
59     return n * (n + 1) // 2
60
61 # Example usage
62 n = 10
63 total_sum = sum_of_natural_numbers(n)
64 print(f"\nSum of the first {n} natural numbers is: {total_sum}")
```

Output :

```
/Documents/Desktop/AI-AS/lab6.1-4054.py

Sum of the first 10 natural numbers is: 55
```

Explanation:

The formula $n(n+1)/2$ is used to calculate the sum of the first n natural numbers efficiently.

Task Description #5: Classes (Bank Account Class)

Scenario

You are designing a basic banking application.

Task

- Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(), and check_balance().
- Analyze the AI-generated class structure and logic.
- Add meaningful comments and explain the working of the code.

Expected Output #5

- Complete Python Bank Account class.
- Demonstration of deposit and withdrawal operations with updated balance.
- Well-commented code with a clear explanation.

```

lab6.1-4054.py
lab6.1-4054.py > ...

67 # Task 5
68 # designing a basic banking application. -> class with methods such as deposit(), withdraw(), and check_balance().
69 class BankAccount:
70     def __init__(self, account_holder, initial_balance=0):
71         self.account_holder = account_holder
72         self.balance = initial_balance
73
74     def deposit(self, amount):
75         if amount > 0:
76             self.balance += amount
77             print(f"Deposited: ${amount:.2f}")
78         else:
79             print("Deposit amount must be positive.")
80
81     def withdraw(self, amount):
82         if amount > 0:
83             if amount <= self.balance:
84                 self.balance -= amount
85                 print(f"Withdrew: ${amount:.2f}")
86             else:
87                 print("Insufficient funds.")
88         else:
89             print("Withdrawal amount must be positive.")
90
91     def check_balance(self):
92         print(f"Current balance: ${self.balance:.2f}")
93
94 # Example usage
95 account = BankAccount("John Doe", 1000)
96 account.check_balance()
97 account.deposit(500)
98 account.withdraw(200)
99 account.check_balance()
100 account.withdraw(2000)

```

Output :

```

/Documents/Desktop/AI-AS/lab6.1-4054.py
Current balance: $1000.00
Current balance: $1000.00
Deposited: $500.00
Withdrew: $200.00
Current balance: $1300.00
Insufficient funds.

```

Explanation:

The BankAccount class represents a simple banking system. It has an initializer that sets the account holder's name and initial balance.

The deposit() method allows adding money to the account, while the withdraw() method allows taking money out, provided there are sufficient funds. The check_balance() method displays the current balance of the account.

--	--

--	--